VeRoLog Solver Challenge: Detailed Solution Description

Domonkos Rózsay, Jia Sidan

December 31, 2025

1 Problem Description

The problem considered is a complex multi-day Vehicle Routing Problem (VRP) with additional constraints concerning tool distribution and collection. It originates from the VeRoLog Solver Challenge and models a scenario where a service provider must manage a limited inventory of reusable tools across a finite planning horizon.

1.1 Key Constraints

A set of customer requests R must be served. Each request $r \in R$ specifies:

- A location (node).
- A specific tool type and quantity.
- A delivery time window $[S_r, E_r]$.
- A usage duration D_r (days).

Once delivered, tools must remain at the customer's location for D_r consecutive days. On the day immediately following this period, all tools must be collected. This creates a strict coupling between the delivery day d_{del} and the pickup day $d_{pick} = d_{del} + D_r + 1$.

1.2 Resource Limitations

The problem is constrained by two major resource types:

1. **Global Tool Inventory:** For each tool type, there is a maximum global quantity available. If tools are at customer sites or on trucks, they are not available at the depot.
2. **Daily Vehicle Limits:** On any given day, vehicles have:
   - Maximum weight capacity.
   - Maximum travel distance.
   - Requirement to start and end at the single depot.

The objective is to minimize a composite cost function including global vehicle costs, daily vehicle usage costs, distance costs, and tool usage costs.

2 Solution Approach

Our strategy decouples the temporal decisions (when to deliver) from the spatial decisions (how to route). This results in a three-phase modular architecture.

2.1 Phase 1: Tool Distribution Planning (CP-SAT)

The first phase addresses the "Tool Locking" problem. Since the total number of tools is limited, serving a request on a specific day removes those tools from the inventory for the duration of the request. We formulate this as a Constraint Satisfaction Problem (CSP) and solve it using Google OR-Tools CP-SAT.

2.1.1 Mathematical Formulation

Let H be the set of days in the planning horizon. Let $x_{r,d}$ be a binary variable equal to 1 if request r is delivered on day d, and 0 otherwise.

Constraints: 1. **Single Service:** Each request must be delivered exactly once within its time window.

$$\sum_{d=S_r}^{E_r} x_{r,d} = 1 \quad \forall r \in R \tag{1}$$

2. **Global Inventory Limit:** For every tool type k and every day $t \in H$, the number of tools currently in use (at customers) must not exceed the total available amount $Limit_k$. A request r uses tools on day t if it was delivered on a day d such that $d \leq t < d + D_r + 1$.

$$\sum_{r \in R_k} \sum_{d=\max(S_r, t-D_r)}^{t} count_r \cdot x_{r,d} \leq Limit_k \quad \forall t \in H, \forall k \tag{2}$$

2.2 Phase 2: Construction of Basic Routes

Once the delivery days are fixed, the problem decomposes into independent daily routing problems. However, simply routing purely greedily day-by-day can lead to deadlocks where critical tools needed for a delivery are stuck at a customer site waiting for pickup.

We implement a constructive heuristic that prioritizes "Critical Pickups."

2.2.1 Construction Logic

For each day t:

1. **Identify Tasks:** Collect all deliveries assigned to day t (from Phase 1) and all pickups resulting from deliveries made on previous days.
2. **Calculate Inventory State:** Determine available tools at the depot.
3. **Route Generation:** We initialize a new vehicle and greedily add visits.
4. **Criticality Check:**
   - We identify "Critical Tools"—tools where the daily demand exceeds current depot inventory.
   - Pickups containing these tools are given extreme priority ($Bonus = 50000$).
   - **Triangle Inequality Check:** Before adding a critical pickup, we verify that the vehicle has enough remaining distance to return to the depot and potentially perform the delivery that requires the tool (simulating a Pickup $\rightarrow$ Depot $\rightarrow$ Delivery flow).
5. **Emergency Mode:** If a vehicle is carrying critical tools but cannot find a "good" delivery, the algorithm forces an "Emergency Delivery" to clear the inventory and make the tools available, even if it adds extra distance.

This phase produces a feasible solution (Basic Routes), ensuring that all daily distance and capacity constraints are met.

2.3 Phase 3: Route Optimization

The initial routes are feasible but often inefficient (e.g., zig-zagging or underutilized vehicles). We apply a Local Search optimizatio.

2.3.1 Local Search Operators

We employ a Variable Neighborhood Descent (VND) approach with the following operators. All operators strictly check if the route is valid before accepting a move.

1. **Move Inside Route (Relocate):** Moves a single customer to a different position within the same route.
2. **Swap Inside Route:** Exchanges the positions of two customers within the same route.
3. **Move Block:** Moves a contiguous sequence of customers (a segment) to a new position. This preserves local optimality of sub-tours.
4. **Reverse Block (2-Opt):** Reverses the order of a segment. This is crucial for untangling crossing edges.
5. **Inter-Route Relocate:** Moves a customer from Route A to Route B. This is the primary mechanism for reducing the number of vehicles. If a route becomes empty (depot $\rightarrow$ depot), it is eliminated, saving the fixed vehicle cost.
6. **Inter-Route Swap:** Exchanges customers between two different routes to balance load or distance.
7. * **More ideas:** We had more ideas that we could not implement in time. The most obvious would be the **exchanging of routes between days** as that could potentially solve problems that occur from the initial distribution planning phase not taking into account the truck constraints.

2.3.2 Cost Function

The optimizer minimizes a weighted objective:

$$Cost = \sum (Distance \times C_{dist}) + (N_{vehicles} \times (C_{day} + C_{global}))$$

This explicitly encourages emptying routes to remove vehicles entirely as we cannot change the underlying distribution at this phase.

3 Case Study: Instance Test 2

To illustrate the impact of our optimization phase, we analyze the routing structure of "Test Instance 2" on Day 2.

3.1 Basic Routes (Phase 2 Output)

The constructive heuristic produced 84 vehicles for the first 10 tracked segments. Many vehicles performed single trips (Depot $\rightarrow$ Customer $\rightarrow$ Depot) due to the greedy nature of the construction.

Listing 1: Basic Routes (Day 2, first 10 trucks)

```
DAY = 2 | NUMBER_OF_VEHICLES = 84
1  R 0 -1 0
2  R 0 7 0
3  R 0 -9 0
4  R 0 -10 0
5  R 0 -14 0
6  R 0 -23 0
7  R 0 26 0
8  R 0 29 0
9  R 0 34 0
10 R 0 -35 0
...
```

3.2 Optimised Routes (Phase 3 Output)

After applying the Local Search operators, the solution was condensed significantly. The number of vehicles dropped drastically (from 84 to 13 in this subset view), and routes became much denser.

Listing 2: Optimised Routes (Day 2)

```
DAY = 2 | NUMBER_OF_VEHICLES = 13
1  R 0 -191 60 178 -180 149 -153 229 -35 204 -67 -84 38 0 137 -163 0
2  R 0 -206 238 -170 289 -14 89 -131 -218 230 242 -159 -216 82 -42 0
3  R 0 39 -10 -23 157 -293 127 -87 275 0
4  R 0 29 -107 -264 182 -135 210 -226 130 -221 0
5  R 0 239 -262 25 183 -1 0
...
```

Analysis:

- **Aggregation:** Route 1 in the optimized set visits 15 nodes, effectively merging what would have been 15 separate trips in the basic construction.
- **Mixed Operations:** The optimized routes efficiently mix Pickups (negative IDs) and Deliveries (positive IDs). For example, Route 1 performs pickups at nodes 191, 180, 153, etc., gathering tools to fulfill the deliveries at nodes 60, 178, 149.
- **Cost Reduction:** This consolidation massively reduces the fixed vehicle costs and the redundant deadheading to/from the depot.

4 Computational Results

We evaluated our algorithm on the standard benchmark sets ("Early", "Hidden", and "Late"). We compared our current modular approach ("New") against our previous baseline implementation ("Old").

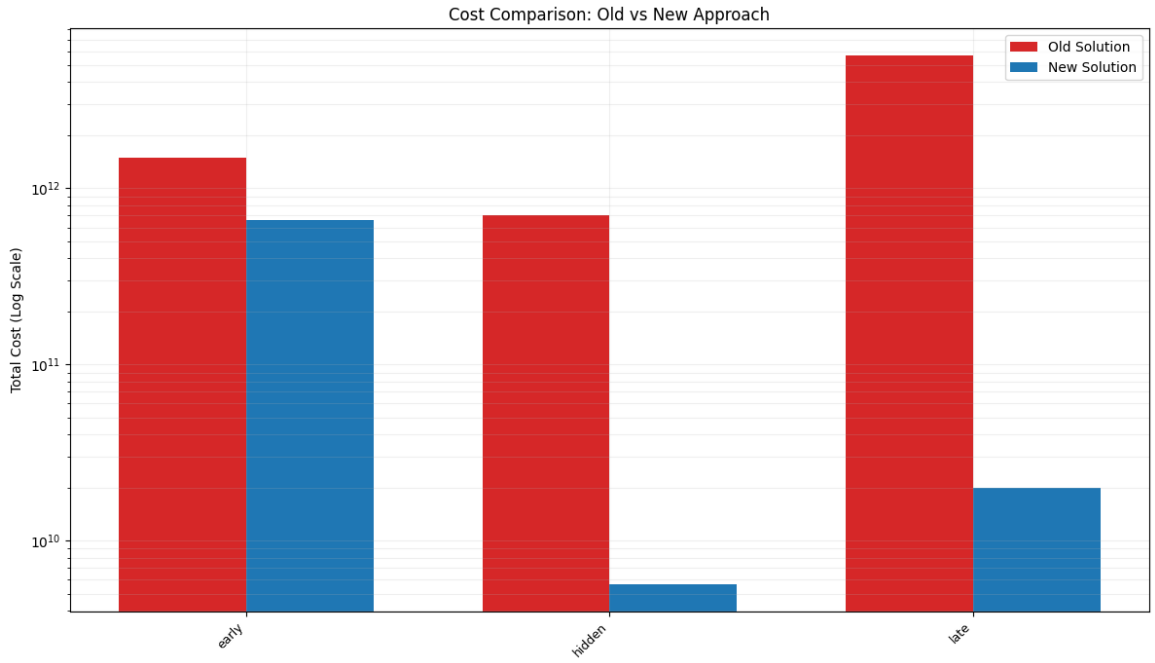

Figure 1: Enter Caption

Figure 2: Cost Comparison between Old and New Solution Approaches across different instance types (early, late, hidden). In hidden and late there are significant improvements, however there were multiple instances in both that we did not find a solution for (2/25 in early, 19/50 in hidden and 5/25 in late). Detailed costs at the end.

The results indicate a significant improvement in solution stability.

- **Feasibility:** The New approach finds feasible solutions for large instances (e.g., 1000 requests over 30 days) where the Old approach often failed (represented by missing bars or non-convergence).
- **Cost Efficiency:** In the "Early" dataset, the New approach consistently achieves lower total costs. For example, on the instance "tools over vehicles... 1000 requests", costs dropped from $\approx 1.2 \times 10^{10}$ to $\approx 9.8 \times 10^{9}$.
- **Robustness:** The handling of "Late" instances, which typically feature tighter time windows and inventory constraints, shows that the Phase 1 CP-SAT model effectively prevents tool starvation.

Note: Instances where the solver could not find a feasible solution within the time limit are excluded from the cost visualization.

5 Conclusion

This hierrchical approach tackles the VeRoLog Solver Challenge by isolating the complex tool-management constraints from the vehicle routing logic. The integration of a CP-SAT solver for temporal planning with a heuristic local search for spatial optimization proves to be robust and efficient. The dramatic reduction in vehicle usage observed in the case study validates the effectiveness of our route optimization operators. There are multiple directions of improvement yet.

6 Appendix

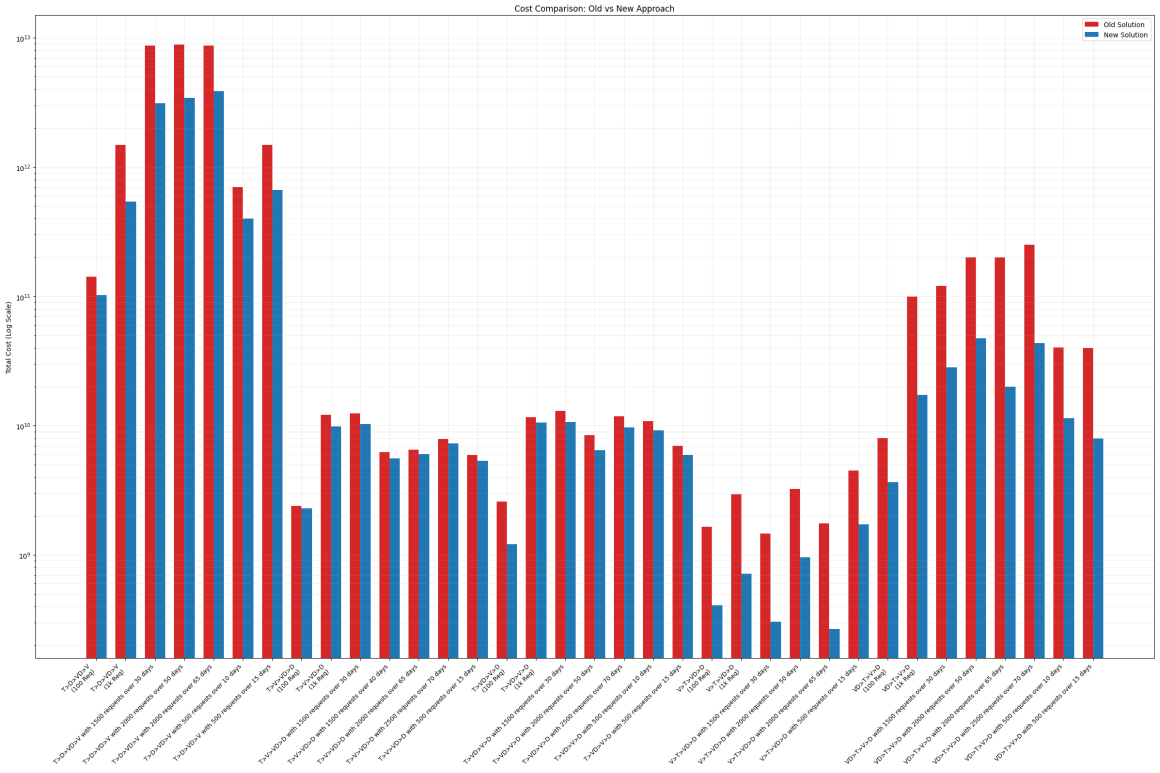

Figure 3: Enter Caption

Figure 4: Detailed Cost Comparison between Old and New Solution Approaches across specific instance types. Name reflects cost structure of that instance type (included in early, late and hidden). Note , that there were multiple instances in hidden that we could not solve.